

STATISTIQUES DESCRIPTIVES avec R

Christophe Ambroise

2025-03-04

Table of contents

1	Le langage R de base (R base)	3
1.1	Qu'est-ce que R ?	3
1.2	Principales fonctionnalités	3
2	Historique de R	3
2.1	Chronologie	3
2.2	Se tenir informé	4
2.3	Qualités et défauts de R	4
2.4	Qualités et défauts de R	5
2.5	Concurrents directs et alternatives	5
2.6	Pléthore de livres sur R	5
2.7	Pléthore de livres sur R	6
3	RStudio : une interface puissante pour R	6
3.1	Fonctionnalités principales	6
3.2	Organisation des panneaux dans RStudio	6
3.3	Interagir avec R dans RStudio	7
3.4	La console R (panneau en bas à gauche)	7
3.5	Éditeur de script (panneau en haut à gauche)	7
3.6	Raccourcis utiles	7
3.7	Commentaires	8
3.8	Astuce: Exécution de segments de votre code	8
3.9	Bloc de code	8
3.10	Utiliser R comme calculatrice	8
3.11	Commande incomplète	9
3.12	Astuce: Annulation des commandes	9
3.13	Ordre des opérations	9
3.14	Fonctions mathématiques	10

3.15	Se souvenir	11
3.16	Comparer les choses	11
3.17	Astuce: Comparer les nombres	12
3.18	Variables et affectation	12
3.19	Noms de variables	13
3.20	Affectation (suite)	14
3.21	Gérer votre environnement	14
3.22	Astuce: objets cachés	14
3.23	Contenu d'une fonction	15
3.24	Suppression	16
3.25	Astuce: Avertissements vs erreurs	16
3.26	R Packages	16
3.27	Exercices	17
4	Chercher de l'aide	17
4.1	Lire les fichiers d'aide	17
4.2	Opérateurs spéciaux	17
4.3	Obtenir de l'aide sur les packages	18
4.4	Quand vous vous souvenez de la fonction	18
5	Mélanger code et texte avec knitr	18
5.1	Motivations: reproductibilité de la recherche	18
5.2	En bref	18
5.3	Référence	19
5.4	Installer knitr	19
5.5	Créer un de type fichier Rmarkdown (.Rmd)	20
5.6	YAML	20
5.7	Markdown	21
5.8	Markdown	21
5.9	Markdown	22
5.10	Compilation	22
5.11	Un peu plus de Markdown	22
5.12	Un peu plus de Markdown	22
5.13	Morceaux de code	23
5.14	Morceaux de code	23
5.15	Knitr en diagramme	24
5.16	Options de blocs	24
5.17	Ressources	24
5.18	Exercice	25
5.19	Pour aller plus loin : Quarto vs Jupyter Notebooks	25
5.20	Jupyter Notebooks	25
5.21	Quarto	25
5.22	Différences Clés	26

1 Le langage R de base (R base)

1.1 Qu'est-ce que R ?

En bref

R est un logiciel de développement scientifique spécialisé dans le calcul et l'**analyse statistique**

Mais aussi

- un langage interprété,
- un environnement de développement,
- un projet open source (projet **GNU**),
- un logiciel multi-plateforme (Linux, Mac, Windows),
- la 18ième lettre de l'alphabet

1.2 Principales fonctionnalités

1. Gestionnaire de données - Lecture, manipulation, stockage.
2. Algèbre linéaire - Opérations classiques sur vecteurs, tableaux et matrices
3. Statistiques et analyse de données - Dispose d'un *grand* nombre de méthodes d'analyse de données (des plus anciennes et aux plus récentes)
4. Moteur de sorties graphiques - Sorties écran ou fichier
5. Système de modules - Alimenté par la communauté (+ de 2000 extensions!)
6. Interface facile avec C/C++, Fortran

2 Historique de R

2.1 Chronologie

Année	Événement
1976	Développement du langage S au Bell Labs par John Chambers et son équipe.
1988	Développement de S+ , version commerciale de S, par Statistical Sciences Inc. (plus tard AT&T).
1993	Création de R , inspiré de S, par Robert Gentleman et Ross Ihaka à l'Université d'Auckland.
1995	Publication des codes sources de R sous licence GNU/GPL .

Année	Événement
1997	Élargissement du groupe de développement avec la participation de Martin Mächler, Douglas Bates et Luke Tierney .
2000	Sortie de R 1.0.0 , première version stable officielle.
2002	Création de la R Foundation for Statistical Computing
2010	Lancement de RStudio , un environnement de développement intégré (IDE) dédié à R.
2015	Introduction du tidyverse , une collection de packages modernisant la manipulation de données et la visualisation.

2.2 Se tenir informé

1. La page web de la Fondation R

- Statuts, liens et références.
- <https://www.r-project.org/foundation/>

2. La page web du CRAN (Comprehensive R Archive Network)

- Binaires d'installation, packages, documentations, etc.
- <https://cran.r-project.org/>

3. Conférences des utilisateurs de R

- **useR! 2025** : La conférence internationale des utilisateurs
 - <https://user2025.r-project.org/>
- **Rencontres R 2025** : Les 11èmes Rencontres R auront lieu du 19 au 21 mai 2025 à l'Université de Mons (UMONS) en Belgique.
 - <https://rr2025.sciencesconf.org/>

4. The R Journal

- Propose des articles sur de nouvelles extensions, des applications.
- <https://journal.r-project.org/>

2.3 Qualités et défauts de R

Avantages

- **Libre et gratuit** : accessible à tous sans coût.
- **Écosystème riche** : des milliers de packages pour la statistique, le machine learning et la visualisation.
- **Large communauté** : support actif et nombreuses ressources en ligne.

- **Langage polyvalent** : adapté à l'analyse de données, la bioinformatique et la finance.
- **Puissant pour les statistiques** : leader dans les méthodes avancées d'analyse.
- **Visualisation avancée** : graphiques élégants (ggplot2...).
- **Reproductibilité** : grâce à R Markdown, Quarto et knitr.
- **Intégration facile** : avec Python, SQL, Spark, et d'autres langages.

2.4 Qualités et défauts de R

Inconvénients

- **Performance** : peut être lent sur des très grands volumes de données.
- **Gestion de la mémoire** : moins optimisée que d'autres langages comme Julia ou Rust.
- **Courbe d'apprentissage** : syntaxe parfois déroutante pour les débutants.
- **Support de l'IA et du deep learning** : moins développé que Python avec TensorFlow ou PyTorch.
- **Personnalisation graphique** : ggplot2 est puissant mais nécessite des ajustements parfois lourds.

2.5 Concurrents directs et alternatives

Domaine	Logiciels concurrents
Algèbre linéaire	Matlab (Mathworks), Scilab (INRIA), Octave (GNU)
Statistiques	SAS (SAS Inc.), SPSS (IBM), Stata
Calcul symbolique	Mathematica (Wolfram), Maple (Maplesoft), Maxima (GNU)
Autres alternatives	Python (le plus sérieux concurrent), Julia (rapide et moderne)

R reste un outil incontournable en **science des données** et **statistiques**, bien que Python et Julia gagnent en popularité pour le machine learning et le calcul haute performance.

2.6 Pléthore de livres sur R

Ouvrages de référence

- **Anciens ouvrages de référence** : [Liste complète](#)
- **R base** : Un livre de base pour débuter en R - [Lire en PDF](#)
- **Introduction à R** (par l'équipe du CRAN) - [Lire en ligne](#)
- **The R Book** (Michael J. Crawley) - [Détails](#)

Data Science et visualisation

- **ggplot2: Elegant Graphics for Data Analysis** (Hadley Wickham) - [Lire sur Bookdown](#)
- **R for Data Science** (Hadley Wickham & Garrett Grolemund) - [Lire en ligne](#)
- **Tidyverse Skills** (Hadley Wickham et al.) - [Explorer sur Bookdown](#)

2.7 Pléthore de livres sur R

R Markdown, Quarto et reporting

- **R Markdown: The Definitive Guide** (Yihui Xie, J.J. Allaire, Garrett Grolemund) - [Lire sur Bookdown](#)
- **Quarto for R Users** - [Documentation officielle](#)
- **Dynamic Documents with R and knitr** (Yihui Xie) - [Détails](#)

Statistiques et Machine Learning

- **Modern Applied Statistics with S** (Venables & Ripley) - [Détails](#)
- **An Introduction to Statistical Learning** (James, Witten, Hastie, Tibshirani) - [Lire gratuitement](#)
- **Elements of Statistical Learning** (Hastie, Tibshirani, Friedman) - [Accès libre](#)
- **Hands-On Machine Learning with R** (Brad Boehmke & Brandon Greenwell) - [Détails](#)

3 RStudio : une interface puissante pour R

3.1 Fonctionnalités principales

RStudio est un **environnement de développement intégré (IDE)** pour R, offrant :

- Un **éditeur de script intégré** avec coloration syntaxique.
- Une compatibilité **multi-plateforme** (Windows, macOS, Linux, serveurs).
- Une **gestion de projet et versionning** facilitée (intégration avec Git).
- Un accès direct aux **packages, graphiques et documentation**.

3.2 Organisation des panneaux dans RStudio

Lorsque vous ouvrez RStudio, l'interface est divisée en **quatre panneaux** :

Panneau	Fonction
Console (en bas à gauche)	Exécution des commandes R en direct.
Environnement / Historique (en haut à droite)	Liste des variables et historique des commandes.
Fichiers / Graphiques / Packages / Aide (en bas à droite)	Gestion des fichiers, affichage des graphiques, installation de packages et accès à la documentation.
Éditeur de script (en haut à gauche, une fois un fichier ouvert)	Écriture et exécution de scripts R.

3.3 Interagir avec R dans RStudio

Il existe **deux principales façons** d'interagir avec R :

1. **Via la console** : idéale pour tester des commandes rapidement.
2. **Via des fichiers de script** (.R) : recommandé pour un travail structuré et reproductible.

3.4 La console R (panneau en bas à gauche)

- C'est l'endroit où **R exécute directement les commandes**.
- Les résultats s'affichent immédiatement après chaque commande.
- **Attention** : les commandes tapées ici sont perdues après la fermeture de la session.

3.5 Éditeur de script (panneau en haut à gauche)

- Permet d'**écrire du code R et de le sauvegarder**.
- Exécuter du code **sans le retaper** à chaque session.
- Envoyer une ligne de code à la console avec **Ctrl + Entrée** (Windows/Linux) ou **Cmd + Entrée** (Mac).

3.6 Raccourcis utiles

Action	Raccourci
Passer de la console à l'éditeur	Ctrl + 1
Passer de l'éditeur à la console	Ctrl + 2
Exécuter une ligne de code	Ctrl + Entrée (Windows/Linux) ou Cmd + Entrée (Mac)

3.7 Commentaires

- Utilisez # signes pour commenter.
- Commentez libéralement dans vos scripts R.
- Tout ce qui se trouve à droite d'un # est ignoré par R.

3.8 Astuce: Exécution de segments de votre code

- RStudio vous offre une grande souplesse dans l'exécution du code depuis l'éditeur fenêtre. Il existe des boutons, des choix de menus et des raccourcis clavier. Pour exécuter la ligne en cours, vous pouvez
1. cliquez sur le bouton Run situé au-dessus du panneau de l'éditeur,
 2. sélectionnez "Run Lines" dans le menu "Code", ou
 3. appuyez sur Ctrl-Entrée dans Windows ou Linux ou sur Commande-Entrée sur OS X. (Ce raccourci peut également être vu en survolant la souris sur le bouton).

3.9 Bloc de code

- Pour lancer un bloc de code, sélectionnez-le, puis Exécuter.
- Si vous avez modifié une ligne de code dans un bloc de code que vous venez d'exécuter, il n'est pas nécessaire de resélectionner la section et Run, vous pouvez utiliser le bouton suivant,
- Ré-exécuter la région précédente Cela exécutera le bloc de code précédent incluant les modifications que vous avez apportées.

3.10 Utiliser R comme calculatrice

La chose la plus simple que vous puissiez faire avec R est l'arithmétique:

```
1 + 100
```

```
[1] 101
```

Et R imprimera la réponse, avec un précédent "[1]". Ne t'inquiète pas pour ça pour l'instant, nous l'expliquerons plus tard. Pour l'instant, considérez-le comme indiquant une sortie.

3.11 Commande incomplète

Comme bash, si vous tapez une commande incomplète, R vous attendra pour compléter l'entrée:

```
> 1 +
```

```
+
```

Chaque fois que vous appuyez sur Entrée et que la session R affiche un “+” au lieu d’un “>”, signifie qu’il attend que vous complétiez la commande. Si vous voulez annuler une commande que vous pouvez simplement appuyer sur “Esc” et RStudio vous rendra le “>” rapide.

3.12 Astuce: Annulation des commandes

- Si vous utilisez R depuis la ligne de commande plutôt que depuis RStudio,
- vous devez utiliser `Ctrl + C` au lieu de `Esc` pour annuler la commande. Ce s’applique également aux utilisateurs de Mac!
- Annuler une commande n’est pas seulement utile pour tuer des commandes incomplètes:
- vous pouvez aussi l’utiliser pour dire à R d’arrêter d’exécuter du code (par exemple si
- en prenant beaucoup plus de temps que prévu, ou pour se débarrasser du code que vous êtes en train d’écrire.

3.13 Ordre des opérations

Lorsque vous utilisez R comme calculatrice, l’ordre des opérations est le même que vous aurait appris à l’école.

De la plus haute à la plus basse préséance:

- Parenthèses: (,)
- Exposants: `^` ou `**`
- Diviser: `/`
- Multiplier: `*`
- Ajouter: `+`
- Soustraire: `-`

```
3 + 5 * 2
```

```
[1] 13
```

Utilisez des parenthèses pour regrouper les opérations afin de forcer l'ordre de évaluation si elle diffère du défaut, ou pour préciser ce que vous avez l'intention

```
(3 + 5) * 2
```

```
[1] 16
```

Cela peut devenir compliqué lorsque cela n'est pas nécessaire, mais clarifie vos intentions. Rappelez-vous que d'autres peuvent lire votre code ultérieurement.

```
(3 + (5 * (2 ^ 2))) # difficile à lire  
3 + 5 * 2 ^ 2 # si vous vous souvenez des règles  
3 + 5 * (2 ^ 2) # Si vous oubliez certaines règles, cela pourrait vous aider
```

Le texte après chaque ligne de code s'appelle un "commentaire". Tout ce qui suit le symbole de hachage (ou octothorpe) "#" est ignoré par R lorsqu'il exécute du code.

Les nombres vraiment petits ou grands ont une notation scientifique:

```
2/10000
```

```
[1] 2e-04
```

Ce qui est un raccourci pour "multiplié par 10^{-XX} ". Donc $2e-4$ est un raccourci pour $2 * 10^{-4}$.

Vous pouvez aussi écrire des nombres en notation scientifique:

```
5e3 # Notez l'absence de moins ici
```

```
[1] 5000
```

3.14 Fonctions mathématiques

R a beaucoup de fonctions mathématiques intégrées. Pour appeler une fonction, nous tapons simplement son nom, suivi par des parenthèses ouvertes et fermantes. Tout ce que nous tapons à l'intérieur des parenthèses s'appelle la fonction arguments:

```
sin (1) # fonctions trigonométriques
```

```
[1] 0.841471
```

```
log (1) # logarithme naturel
```

```
[1] 0
```

```
log10 (10) # base-10 logarithme
```

```
[1] 1
```

```
exp (0.5) #  $e^{(1/2)}$ 
```

```
[1] 1.648721
```

3.15 Se souvenir

Ne vous souciez pas d'essayer de vous souvenir de toutes les fonctions de R:

- rechercher sur Google,
- ou si vous vous souvenez de la début du nom de la fonction, utilisez *complétion* dans RStudio.

C'est un avantage que RStudio sur R , il dispose de *capacités d'auto-complétion* qui vous permettent plus facilement rechercher des fonctions, leurs arguments et les valeurs qu'ils prendre.

Taper un ? Avant le nom d'une commande ouvrira la page d'aide pour cette commande. En plus de fournir

- une description la page d'aide affichera généralement
- une collection d'exemples

3.16 Comparer les choses

Nous pouvons également faire la comparaison en R:

```
1 == 1 # égalité (noter deux signes égaux, lire comme "est égal à")
```

```
[1] TRUE
```

```
1 != 2 # inégalité (lire comme "n'est pas égal à")
```

```
[1] TRUE
```

```
1 < 2 # moins que
```

```
[1] TRUE
```

```
1 <= 1 # inférieur ou égal à
```

```
[1] TRUE
```

```
1 > 0 # plus grand que
```

```
[1] TRUE
```

```
1 >= -9 # supérieur ou égal à
```

```
[1] TRUE
```

3.17 Astuce: Comparer les nombres

- Un mot d'avertissement sur la comparaison des chiffres: vous devriez ne jamais utiliser `==` pour comparer deux nombres à moins qu'ils ne soient entiers (un type de données pouvant représenter spécifiquement uniquement des nombres entiers).
- Les ordinateurs ne peuvent représenter que des nombres décimaux avec un certain degré de précision, donc deux nombres qui semblent le même lorsqu'il est imprimé par R, peut effectivement avoir différentes représentations sous-jacentes et donc être différent par une petite marge d'erreur (appelée Machine tolérance numérique).
- Au lieu de cela, vous devez utiliser la fonction `all.equal`.
- Lectures complémentaires: <http://floating-point-gui.de/>

3.18 Variables et affectation

Stocker des valeurs dans des variables en utilisant l'opérateur d'affectation `<-`

```
x <- 1/40
```

```
x
```

```
[1] 0.025
```

Plus précisément, la valeur stockée est une approximation * décimale * de cette fraction appelée [nombre à virgule flottante] (http://en.wikipedia.org/wiki/Floating_point).

Recherchez l'onglet **Environment** dans l'un des volets de RStudio, et vous verrez que `x` et sa valeur est apparu. Notre variable `x` peut être utilisée à la place d'un nombre dans tout calcul qui attend un nombre:

```
log (x)
```

```
[1] -3.688879
```

Notez également que les variables peuvent être réaffectées:

```
x <- 100
```

`x` contenait la valeur 0.025 et a maintenant la valeur 100.

Les valeurs d'affectation peuvent contenir la variable affectée à:

```
x <- x + 1 #notice comment RStudio met à jour sa description de x en haut à droite
```

Le côté droit de l'affectation peut être toute expression R valide. Le côté droit est *entièrement évalué* avant que l'affectation ait lieu.

3.19 Noms de variables

Les noms de variables peuvent contenir des lettres, des chiffres, des traits de soulignement et des points. Ils ne peuvent pas commencer avec un nombre ni contenir des espaces du tout. Différentes personnes utilisent différentes conventions pour les noms de variables longues, celles-ci comprennent

- periods.between.words
- souligne_entre_mots
- camelCaseToSeparateWords

Ce que vous utilisez dépend de vous, mais **soyez cohérent**.

3.20 Affectation (suite)

Il est également possible d'utiliser l'opérateur = pour l'affectation:

```
x = 1/40
```

Mais c'est beaucoup moins courant parmi les utilisateurs de R.

Donc, la recommandation est d'utiliser <=.

3.21 Gérer votre environnement

Il existe quelques commandes utiles que vous pouvez utiliser pour interagir avec la session R.

ls listera toutes les variables et fonctions stockées dans l'environnement global (votre session de travail):

```
ls()
```

```
[1] "x"
```

3.22 Astuce: objets cachés

- Comme dans le shell, ls cachera toutes les variables ou fonctions commençant avec un "." par défaut.
- Pour lister tous les objets, tapez ls (all.names = TRUE)

Remarque

Notez ici que nous n'avons donné aucun argument à ls, mais nous avons quand même nécessaire de donner aux parenthèses de dire à R d'appeler la fonction.

3.23 Contenu d'une fonction

Si vous tapez `ls` par lui-même, R imprimera le code source de cette fonction!

```
ls
```

```
function (name, pos = -1L, envir = as.environment(pos), all.names = FALSE,
  pattern, sorted = TRUE)
{
  if (!missing(name)) {
    pos <- tryCatch(name, error = function(e) e)
    if (inherits(pos, "error")) {
      name <- substitute(name)
      if (!is.character(name))
        name <- deparse(name)
      warning(gettextf("%s converted to character string",
        sQuote(name)), domain = NA)
      pos <- name
    }
  }
  all.names <- .Internal(ls(envir, all.names, sorted))
  if (!missing(pattern)) {
    if ((ll <- length(grep("[", pattern, fixed = TRUE))) &&
      ll != length(grep("]", pattern, fixed = TRUE))) {
      if (pattern == "[") {
        pattern <- "\\["
        warning("replaced regular expression pattern '[' by '\\\\['")
      }
      else if (length(grep("[^\\\\\\\\]\\\\[<-", pattern))) {
        pattern <- sub("\\[<-", "\\\\\\\\[<-", pattern)
        warning("replaced '['<- by '\\\\\\\\[<- in regular expression pattern")
      }
    }
    grep(pattern, all.names, value = TRUE)
  }
  else all.names
}
<bytecode: 0x1455a4ea0>
<environment: namespace:base>
```

3.24 Supression

Vous pouvez utiliser `rm` pour supprimer des objets dont vous n'avez plus besoin:

```
rm (x)
```

Si vous avez beaucoup de choses dans votre environnement et souhaitez les supprimer toutes, vous pouvez transmettre les résultats de `ls` à la fonction `rm`:

```
rm(list=ls())
```

Dans ce cas, nous avons spécifié que les résultats de `ls` devraient être utilisés pour le L'argument `list` dans `rm`. Lorsque vous attribuez des valeurs aux arguments par nom, vous *devez* utiliser l'opérateur `=` !!

Si au lieu de cela nous utilisons `<-`, il y aura des effets secondaires inattendus, ou vous pourriez avoir un message d'erreur:

```
rm(list <- ls ())
```

```
Error in rm(list <- ls()): ... must contain names or character strings
```

3.25 Astuce: Avertissements vs erreurs

- Quand R fait quelque chose d'inattendu! Les erreurs, comme ci-dessus, sont émises lorsque R ne peut pas procéder à un calcul.
- En revanche, les avertissements signifient généralement que la fonction a été exécutée, mais cela n'a probablement pas fonctionné comme prévu.
- Dans les deux cas, le message que R imprime vous donne généralement des indices sur la manière de résoudre un problème.

3.26 R Packages

Il est possible d'ajouter des fonctions à R en écrivant un paquet, ou par obtenir un paquet écrit par quelqu'un d'autre. Au moment de l'écriture, il y a sont plus de 7 000 paquets disponibles sur CRAN (l'archive complète de R réseau). R et RStudio ont des fonctionnalités pour gérer les paquets:

- Vous pouvez voir quels paquets sont installés en tapant

```
installed.packages ()
```

- Vous pouvez installer des paquets en tapant

`install.packages (" packagename ")`, où `packagename` est le nom du package, entre guillemets.

- Vous pouvez mettre à jour les paquets installés en tapant `update.packages ()`
- Vous pouvez supprimer un paquet avec `remove.packages (" packagename ")`
- Vous pouvez rendre un paquet disponible pour être utilisé avec `library (packagename)`

3.27 Exercices

1. Nommer toutes les panels de R studio
2. Réaliser une addition et une multiplication dans la console
3. Sauver un script R avec votre code d'addition

4 Chercher de l'aide

4.1 Lire les fichiers d'aide

R, et chaque paquet, fournissent des fichiers d'aide pour les fonctions. Pour chercher de l'aide sur une fonction d'une fonction spécifique qui est dans un package chargé

```
?nom_fonction  
help(nom_fonction)
```

Cela chargera une page d'aide dans RStudio (ou du texte brut dans R seul).

4.2 Opérateurs spéciaux

Pour demander de l'aide sur des opérateurs spéciaux, utilisez des guillemets:

```
? "+"
```

4.3 Obtenir de l'aide sur les packages

De nombreux paquets contiennent des “vignettes”: des tutoriels et des exemples de documentation. Sans aucun argument, `vignette()` listera toutes les vignettes pour tous les paquets installés;

```
vignette (package =" nom-du-paquet ")
```

listera toutes les vignettes disponibles pour

```
package-name et vignette ("vignette-name")
```

ouvriront la vignette spécifiée.

Si un paquet ne contient aucune vignette, vous pouvez généralement trouver de l'aide en tapant

```
help("nom-du-paquet").
```

4.4 Quand vous vous souvenez de la fonction

Si vous ne savez pas exactement dans quel paquet est une fonction ou comment elle est spécifiquement orthographiée, vous pouvez effectuer une recherche floue:

```
??nom_fonction
```

5 Mélanger code et texte avec knitr

5.1 Motivations: reproductibilité de la recherche

Principe de Claerbout (Géophysicien, Stanford)

An article about computational science in a scientific publication is not the scholarship itself, it is merely advertising of the scholarship. The actual scholarship is the complete software development environment and the complete set of instructions which generated the figures.

5.2 En bref

`knitr`

pour générer des rapports qui combinent le texte, le code et les résultats.

Markdown

pour mettre en forme le texte

Markdown est un langage de balisage léger créé par John Gruber en 2004. Son but est d'offrir une syntaxe facile à lire et à écrire. Un document balisé par Markdown peut être lu en l'état sans donner l'impression d'avoir été balisé ou formaté par des instructions particulières. — Wikipedia

[Lien Wikipedia](#)

Code Chunks

code dans des blocs délimités par des guillemets triples suivis de `{r}`.

5.3 Référence

Rmarkdown est un univers extensible, si vous voulez continuer, lisez

<https://rmarkdown.rstudio.com/>

5.4 Installer knitr

Dans la console

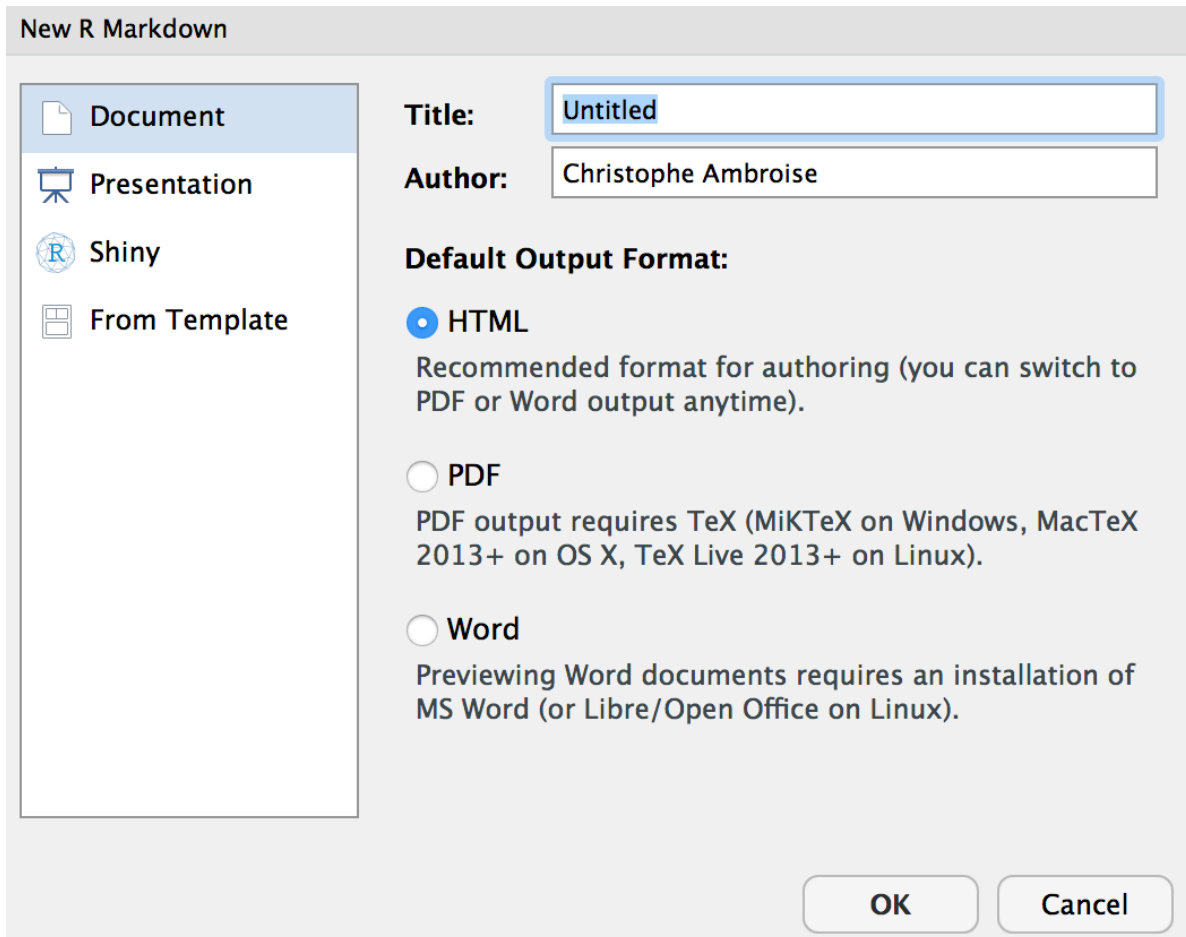
```
install.packages('knitr')
```

Par le menu

Tools -> Install Packages

5.5 Créer un de type fichier Rmarkdown (.Rmd)

Dans R Studio, cliquez sur Fichier → Nouveau fichier → R Markdown et vous obtiendrez une boîte de dialogue du type



The image shows the 'New R Markdown' dialog box in R Studio. On the left, there is a sidebar with four options: 'Document' (selected), 'Presentation', 'Shiny', and 'From Template'. On the right, there are input fields for 'Title' (containing 'Untitled') and 'Author' (containing 'Christophe Ambroise'). Below these is the 'Default Output Format' section with three radio buttons: 'HTML' (selected), 'PDF', and 'Word'. Each radio button has a descriptive text below it. At the bottom right, there are 'OK' and 'Cancel' buttons.

New R Markdown

Document

Title: Untitled

Author: Christophe Ambroise

Default Output Format:

☒ **HTML**
Recommended format for authoring (you can switch to PDF or Word output anytime).

☐ **PDF**
PDF output requires TeX (MiKTeX on Windows, MacTeX 2013+ on OS X, TeX Live 2013+ on Linux).

☐ **Word**
Previewing Word documents requires an installation of MS Word (or Libre/Open Office on Linux).

OK **Cancel**

5.6 YAML

Vous créez un fichier avec une entête dite yaml du type

```
---  
title: "Initial R Markdown document"  
author: "Karl Broman"  
date: "April 23, 2015"  
output: html_document  
---
```

qui précise comment peut être transformé le fichier

5.7 Markdown

Markdown est un langage à balise

- écrivez **en gras** en utilisant deux astérisques, comme ceci: **`**gras**`**,
- écrivez en *italics* en utilisant des traits de soulignement, comme ceci: `_italics_`.

Vous pouvez créer une liste à puces en écrivant une liste avec des tirets ou astérisques, comme ceci:

```
* gras avec double astérisque
* italiques avec des soulignés
* police de type code avec backticks
```

5.8 Markdown

ou comme ça:

```
- gras avec double astérisque
- italiques avec des soulignés
- police de type code avec backticks
```

Vous pouvez créer une liste numérotée en utilisant simplement des chiffres. Vous pouvez utiliser le même nombre encore et encore si vous voulez:

```
1. gras avec double astérisque
1. italiques avec des soulignés
1. police de type code avec backticks
```

Cela apparaîtra comme:

1. gras avec double astérisque
2. italiques avec des soulignés
3. police de type code avec backticks

5.9 Markdown

Vous pouvez créer des en-têtes de section de différentes tailles en initiant une ligne avec un certain nombre de symboles #:

```
# Titre
## Section principale
### Sous-section
#### Sous-sous-section
```

5.10 Compilation

Vous *compilez* le document R Markdown en cliquant sur le “Knit HTML” en haut à gauche.

5.11 Un peu plus de Markdown

- Vous pouvez créer un hyperlien comme celui-ci:

[texte à afficher] (<http://the-web-page.com>).

- Vous pouvez inclure un fichier image comme ceci:

![Caption] (<http://url/for/file>)

Vous pouvez faire des indices (par exemple, F_2) avec F_2 et des exposants (par exemple, F^2) avec F^2 .

5.12 Un peu plus de Markdown

Si vous savez écrire des équations dans [LaTeX] (<http://www.latex-project.org/>), vous serez heureux de savoir que vous pouvez utiliser $E = mc^2$ et $y = \mu + \sum_{i=1}^p \beta_i x_i + \epsilon$ pour insérer des équations mathématiques, comme $E = mc^2$ et

$y = \mu + \sum_{i=1}^p \beta_i x_i + \epsilon$

$$y = \mu + \sum_{i=1}^p \beta_i x_i + \epsilon$$

5.13 Morceaux de code

Markdown est intéressant et utile, mais le plus grand intérêt vient du mélange du texte balisé avec des morceaux de code R.

- Quand traité, le code R sera exécuté; s'ils produisent des valeurs, figures, ceux-ci seront insérés dans le document final.

Les morceaux de code ressemblent à ceci:

```
```{r load_data}
gapminder <- read.csv("~/Desktop/gapminder.csv")
```
```

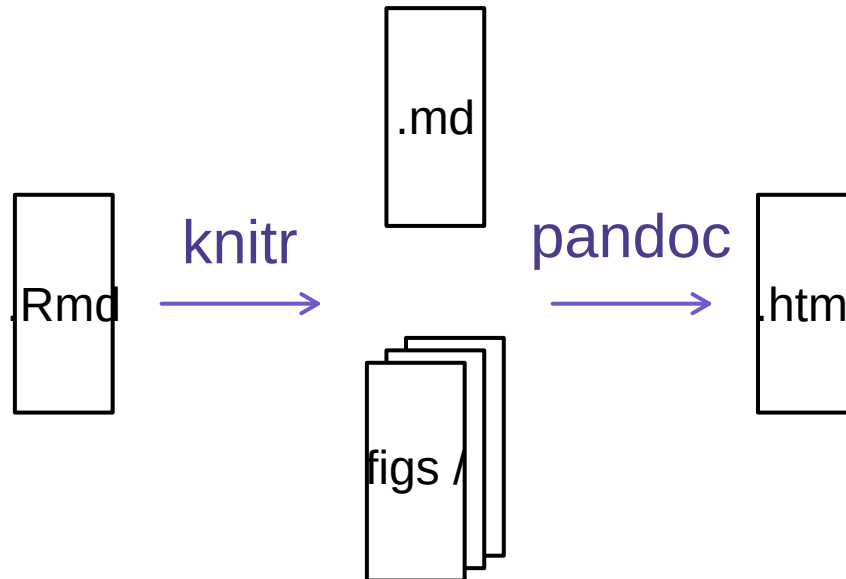
C'est une bonne idée de donner chaque morceau un nom, car ils vous aideront à corriger les erreurs et, si des graphiques sont produit, les noms de fichiers sont basés sur le nom du bloc de code les a produites.

5.14 Morceaux de code

Lorsque vous appuyez sur le bouton “Knit HTML”, le document R Markdown est traité par [knitr] (<http://yihui.name/knitr>) et un simple Markdown document est produit (ainsi que, potentiellement, un ensemble de fichiers de figure): le code R est exécuté et remplacé à la fois par l'entrée et la sortie; si les chiffres sont produits, des liens vers ces chiffres sont inclus.

Les documents Markdown et figure sont ensuite traités par l'outil [pandoc](#), qui convertit le fichier Markdown en fichier html, avec les chiffres incorporés.

5.15 Knitr en diagramme



5.16 Options de blocs

Il y a une variété d'options pour affecter la façon dont les morceaux de code sont traités.

- Utilisez `echo = FALSE` pour éviter que le code lui-même ne soit affiché.
- Utilisez `results = "hide"` pour éviter d'imprimer des résultats.
- Utilisez `eval = FALSE` pour que le code soit affiché mais pas évalué.
- Utilisez `warning = FALSE` et `message = FALSE` pour masquer les avertissements ou messages produits.
- Utilisez `fig.height` et `fig.width` pour contrôler la taille des figures produit (en pouces).

5.17 Resources

Voici une sélection de ressources essentielles pour utiliser R Markdown et knitr :

- **Knitr in a Knutshell Tutorial** : Introduction rapide à knitr par Karl Broman - [Lire ici](#)
- **Dynamic Documents with R and knitr** (Yihui Xie) : Livre détaillé sur la génération de rapports dynamiques - [Détails](#)
- **R Markdown Documentation** : Guide officiel pour R Markdown - [Consulter ici](#)
- **R Markdown Cheat Sheet** : Fiche récapitulative des principales commandes - [Télécharger](#)
- **Quarto Documentation** : Guide officiel pour Quarto, l'évolution de R Markdown - [Lire ici](#)

Ces ressources vous aideront à **exploiter pleinement R Markdown et knitr** pour créer des documents dynamiques et reproductibles !

5.18 Exercice

1. Créer un fichier Rmarkdown qui produira une sortie html avec un code chunk de votre choix

5.19 Pour aller plus loin : Quarto vs Jupyter Notebooks

Knitr et Rmarkdown sont relativement spécifiques à R.

Quarto et les Jupyter Notebooks sont deux outils puissants utilisés pour la création de documents reproductibles mêlant code, texte et visualisations.

5.20 Jupyter Notebooks

- **Nature Interactive** : Interface web interactive permettant d'exécuter du code cellule par cellule, facilitant l'exploration et l'analyse de données en temps réel.
- **Format de Fichier** : Extension `.ipynb`, stockant le code, le texte et les sorties (graphiques) dans un fichier JSON.
- **Langages Supportés** : Principalement Python, mais supporte aussi R, Julia, et d'autres via des noyaux (kernels).
- **Cas d'Utilisation** : Idéal pour le prototypage rapide, l'analyse exploratoire de données et les démonstrations éducatives.

5.21 Quarto

- **Polyvalence** : Plateforme de publication scientifique et technique permettant de créer des articles, blogs, livres et présentations.
- **Indépendance du Langage** : Supporte nativement Python, R, Julia et JavaScript.
- **Contrôle de Version Amélioré** : Fichiers `.qmd` en texte simple, mieux adaptés au contrôle de version avec Git.
- **Rendu et Publication** : Utilise Pandoc pour exporter vers divers formats (HTML, PDF, Word). Exécute le code lors du rendu pour garantir des sorties à jour.

5.22 Différences Clés

- **Interactivité vs. Publication** : Jupyter favorise l'expérimentation en temps réel, tandis que Quarto est conçu pour créer des documents publiables.
- **Gestion du Code et des Sorties** : Jupyter intègre les sorties dans le fichier, rendant le contrôle de version difficile. Quarto exécute le code à chaque rendu sans stocker les sorties.
- **Flexibilité des Formats** : Quarto prend en charge de nombreux formats de sortie et propose des fonctionnalités avancées (références croisées, bibliographies, mises en page personnalisées).